

A Gentle Introduction to Lean: Values, Types, and Other Goodies

2.4.2 List Comprehensions and Proofs

Antonio Eduardo Costa Pereira

September 4, 2026

Contents

1	2.4.2 List Comprehensions and Proofs	1
1.1	One quicksort for arrays and file arrays	2
1.1.1	Adapting an ordinary array	4
1.1.2	Representing a file as an array	4
1.1.3	Creating, copying, and quicksorting the names file . .	7

1 2.4.2 List Comprehensions and Proofs

The preceding quicksort was declared `partial` because Lean had not been given a termination argument. Every recursive function must either be structurally recursive or have a measure that becomes smaller at each recursive call.

Filtering never makes a list longer. The library theorem `List.length_filter_le` states precisely this fact:

```
example (p :  $\alpha \rightarrow \text{Bool}$ ) (xs : List  $\alpha$ ) :  
  (xs.filter p).length  $\leq$  xs.length := by  
  exact List.length_filter_le p xs
```

A recursive call in quicksort begins with a nonempty list `x :: xs` and continues with a filtered part of `xs`. The filtered list has length at most `xs.length`, while `x :: xs` has length `xs.length + 1`. Thus the filtered list is strictly smaller than the original nonempty list:

```
example (x :  $\alpha$ ) (xs : List  $\alpha$ ) (p :  $\alpha \rightarrow \text{Bool}$ ) :
  (xs.filter p).length < (x :: xs).length := by
  exact Nat.lt_succ_of_le (List.length_filter_le p xs)
```

Here `List.length_filter_le p xs` supplies the non-strict inequality

`(xs.filter p).length  $\leq$  xs.length`

and `Nat.lt_succ_of_le` turns it into the strict inequality needed for a recursive call whose original input is `x :: xs`.

The same proof can be attached to both recursive branches of quicksort. The `termination_by` clause tells Lean to use the input list's length as the decreasing measure.

```
def quicksortByProof [Ord  $\alpha$ ] (input : List  $\alpha$ ) : List  $\alpha$  :=
  match input with
  | [] => []
  | x :: xs =>
    let smaller := xs.filter (fun y => compare y x == .lt)
    let remaining := xs.filter (fun y => compare y x != .lt)
    have smaller_decreases : smaller.length < (x :: xs).length := by
      exact Nat.lt_succ_of_le (List.length_filter_le _ _)
    have remaining_decreases : remaining.length < (x :: xs).length := by
      exact Nat.lt_succ_of_le (List.length_filter_le _ _)
    quicksortByProof smaller ++ [x] ++ quicksortByProof remaining
  termination_by input.length

#eval IO.println "\nQuicksort with a termination proof:"
#eval quicksortByProof [7, 1, 4, 1, 9, 2, 6]
```

The two named proofs are used automatically when Lean checks the recursive calls. This definition is total: `partial` is no longer needed.

1.1 One quicksort for arrays and file arrays

An ordinary array and a fixed-record file share the operations needed by quicksort. Both have a size, an operation that reads the item at an index, and an operation that stores an item at an index. The following structure captures that common interface. The type constructor `m` allows the operations to be state transformations for an in-memory array or IO operations for a file.

For demonstration only. Don't try to understand these snippets unless you already know Lean well.

```
structure IndexedStore (m : Type → Type) (α : Type) where
  size : Nat
  aref : Nat → m α
  store : Nat → α → m Unit

partial def swapAt [Monad m]
  (data : IndexedStore m α) (i j : Nat) : m Unit := do
  let left ← data.aref i
  let right ← data.aref j
  data.store i right
  data.store j left

partial def partitionIndexed [Monad m] [Ord α]
  (data : IndexedStore m α) (high scan boundary : Nat) : m Nat := do
  if scan < high then
    let pivot ← data.aref high
    let current ← data.aref scan
    if compare current pivot == .lt then
      swapAt data scan boundary
      partitionIndexed data high (scan + 1) (boundary + 1)
    else
      partitionIndexed data high (scan + 1) boundary
  else
    swapAt data boundary high
    return boundary

partial def quicksortIndexedRange [Monad m] [Ord α]
  (data : IndexedStore m α) (low high : Nat) : m Unit := do
  if low < high then
    let pivotIndex ← partitionIndexed data high low low
    if pivotIndex != 0 then
      quicksortIndexedRange data low (pivotIndex - 1)
      quicksortIndexedRange data (pivotIndex + 1) high

def quicksortIndexed [Monad m] [Ord α]
  (data : IndexedStore m α) : m Unit := do
  if data.size != 0 then
```

```
quicksortIndexedRange data 0 (data.size - 1)
```

This is the only quicksort algorithm in the demonstration. It receives `size`, `aref`, and `store` through `IndexedStore`. Its partition step chooses the last item as the pivot, moves smaller items before it, and returns the pivot's final index. The recursive definitions are `partial` because this demonstration does not supply termination proofs.

1.1.1 Adapting an ordinary array

`StateM` carries the changing array invisibly from one `aref` or `store` operation to the next. This makes an ordinary Lean `Array` implement the common interface.

```
def arrayAsIndexedStore [Inhabited  $\alpha$ ]
  (size : Nat) : IndexedStore (StateM (Array  $\alpha$ ))  $\alpha$  where
  size := size
  aref index := fun array => (array[index]!, array)
  store index value := fun array => ((), array.set! index value)

def quicksortArray [Ord  $\alpha$ ] [Inhabited  $\alpha$ ] (items : Array  $\alpha$ ) : Array  $\alpha$  :=
  let operation := quicksortIndexed (arrayAsIndexedStore items.size)
  (operation.run items).2

#eval IO.println "\nA quicksorted array of natural numbers:"
#eval quicksortArray #[8, 3, 1, 7, 3, 2]

#eval IO.println "\nA quicksorted array of proper names:"
#eval quicksortArray #["Grace", "Ada", "Edsger", "Barbara", "Alan"]
```

`quicksortArray` runs the generic quicksort with the ordinary-array adapter and returns the final `StateM` array.

1.1.2 Representing a file as an array

The file begins with a fixed 64-byte header. The header identifies the format and records the size and number of data slots. A file with 64-byte names and 12 slots begins conceptually as follows; spaces occupy the unused bytes of the header and every short name slot.

LEAN-FILE-ARRAY

```

slot-size=64
slot-count=12
[64-byte name slot 0]
[64-byte name slot 1]
...

```

The metadata at the top lets `FileArray.open` recover the layout from the file itself. The `FileArray` value retains that information for later accesses.

```

def fileArrayHeaderBytes : Nat := 64

structure FileArray where
  path : System.FilePath
  slotSize : Nat
  slotCount : Nat
  deriving Repr

def spaces (count : Nat) : String :=
  String.ofList (List.replicate count ' ')

def padToBytes (width : Nat) (text : String) : IO String := do
  let used := text.toUTF8.size
  if used > width then
    throw <| IO.userError s!"text exceeds its {width}-byte slot: {text}"
  return text ++ spaces (width - used)

def metadataText (slotSize slotCount : Nat) : String :=
  s!"LEAN-FILE-ARRAY\nslot-size={slotSize}\nslot-count={slotCount}\n"

def valueAfterEquals (line : String) : IO Nat :=
  match line.splitOn "=" with
  | [_key, value] =>
    match value.trimAscii.toString.toNat? with
    | some number => return number
    | none => throw <| IO.userError s!"invalid number in header: {line}"
  | _ => throw <| IO.userError s!"invalid header line: {line}"

def parseFileArrayHeader
  (path : System.FilePath) (header : String) : IO FileArray := do
  let lines := header.trimAscii.toString.splitOn "\n"

```

```

match lines with
| magic :: slotLine :: countLine :: _ =>
    if magic != "LEAN-FILE-ARRAY" then
        throw <| IO.userError "not a Lean file array"
    let slotSize ← valueAfterEquals slotLine
    let slotCount ← valueAfterEquals countLine
    return { path, slotSize, slotCount }
| _ => throw <| IO.userError "incomplete Lean file-array header"

```

Lean's standard file handle supports sequential access but has no arbitrary byte-seek operation. For demonstration on Unix, the low-level functions below invoke `dd` to access an exact byte range. The operating-system details remain under the file-array abstraction.

```

def runDD
  (arguments : Array String) (input : Option String := none) : IO String := do
  let result ← IO.Process.output { cmd := "dd", args := arguments } input
  if result.exitCode != 0 then
    throw <| IO.userError result.stderr
  return result.stdout

def readBytesAt
  (path : System.FilePath) (offset count : Nat) : IO String :=
  runDD #[s!"if={path}", "bs=1", s!"skip={offset}", s!"count={count}",
    "status=none"]

def writeBytesAt
  (path : System.FilePath) (offset : Nat) (text : String) : IO Unit := do
  let result ← runDD
    #[s!"of={path}", "bs=1", s!"seek={offset}",
      s!"count={text.toUTF8.size}", "conv=notrunc", "status=none"]
    (some text)
  if !result.isEmpty then
    throw <| IO.userError "dd unexpectedly wrote to standard output"

def FileArray.open (path : System.FilePath) : IO FileArray := do
  let header ← readBytesAt path 0 fileArrayHeaderBytes
  parseFileArrayHeader path header

def FileArray.slotOffset (file : FileArray) (index : Nat) : Nat :=

```

```
fileArrayHeaderBytes + index * file.slotSize
```

The file versions of `aref` and `store` mimic indexed array access. They check the index and ensure that a stored value occupies exactly one slot, so storing one name cannot shift or overwrite neighboring names.

```
def FileArray.aref (file : FileArray) (index : Nat) : IO String := do
  if index >= file.slotCount then
    throw <| IO.userError s!"file-array index out of bounds: {index}"
  let slot ← readBytesAt file.path (file.slotOffset index) file.slotSize
  if slot.toUTF8.size != file.slotSize then
    throw <| IO.userError s!"short file-array slot: {index}"
  return slot

def FileArray.store
  (file : FileArray) (index : Nat) (value : String) : IO Unit := do
  if index >= file.slotCount then
    throw <| IO.userError s!"file-array index out of bounds: {index}"
  if value.toUTF8.size != file.slotSize then
    throw <| IO.userError s!"value does not occupy {file.slotSize} bytes"
  writeBytesAt file.path (file.slotOffset index) value

def FileArray.asIndexedStore (file : FileArray) : IndexedStore IO String where
  size := file.slotCount
  aref := file.aref
  store := file.store
```

`FileArray.asIndexedStore` is the adapter. It passes the file's `aref` and `store` functions to the same `quicksortIndexed` used for ordinary arrays. Quicksort has no knowledge that these operations access a file.

1.1.3 Creating, copying, and quicksorting the names file

The small arrays below are only pools from which random first and family names are chosen. Each generated name is written immediately to a fixed-size file slot. The names file is never loaded into an array or list.

```
def firstNames : Array String :=
  #["Ada", "Alan", "Barbara", "Edsger", "Grace", "John", "Katherine",
    "Margaret"]
```

```

def familyNames : Array String :=
  #["Backus", "Dijkstra", "Hamilton", "Hopper", "Johnson", "Lovelace",
    "Turing", "Wilkes"]

def randomProperName : IO String := do
  let firstIndex ← IO.rand 0 (firstNames.size - 1)
  let familyIndex ← IO.rand 0 (familyNames.size - 1)
  return s!"{firstNames[firstIndex]!} {familyNames[familyIndex]!}"

def writeRandomNameSlots
  (handle : IO.FS.Handle) (slotSize : Nat) : Nat → IO Unit
| 0 => handle.flush
| count + 1 => do
  let name ← randomProperName
  handle.putStr (← padToBytes slotSize name)
  writeRandomNameSlots handle slotSize count

def createRandomNamesFile
  (path : System.FilePath) (slotSize slotCount : Nat) : IO Unit := do
  let handle ← IO.FS.Handle.mk path .write
  handle.putStr
    (← padToBytes fileArrayHeaderBytes (metadataText slotSize slotCount))
  writeRandomNameSlots handle slotSize slotCount

  Before sorting, the original file is copied to another file. The copy is
  streamed in chunks of at most 4096 bytes, so the copy operation's memory
  use does not grow with the file.

partial def copyRemaining
  (source destination : IO.FS.Handle) : IO Unit := do
  let bytes ← source.read 4096
  if bytes.isEmpty then
    destination.flush
  else
    destination.write bytes
    copyRemaining source destination

def copyFileByChunks
  (sourcePath destinationPath : System.FilePath) : IO Unit := do
  let source ← IO.FS.Handle.mk sourcePath .read

```



```
let destination ← IO.FS.Handle.mk destinationPath .write
copyRemaining source destination
```

Only a display routine remains file-specific. It reads one slot at a time and removes its padding for printing.

```
partial def printFileArray (file : FileArray) (index : Nat := 0) : IO Unit := do
  if index < file.slotCount then
    IO.println (← file.aref index).trimAsciiEnd.toString
    printFileArray file (index + 1)
```

```
def originalNamesPath : System.FilePath := "proper_names-original.arr"
def sortedNamesPath : System.FilePath := "proper_names-sorted.arr"
```

```
#eval do
  createRandomNamesFile originalNamesPath 64 12
  copyFileByChunks originalNamesPath sortedNamesPath
  let file ← FileArray.open sortedNamesPath
  quicksortIndexed file.asIndexedStore
  IO.println s!"\n\nThe original file is {originalNamesPath}."
  IO.println s!"Its in-place quicksorted copy is {sortedNamesPath}:"
  printFileArray file
```

Running the tangled tutorial file creates two fixed-record files. The first retains the random order. The second begins as an exact copy and is then quicksorted in place through `FileArray.aref` and `FileArray.store`. With its 64-byte header and twelve 64-byte data slots, its size stays $64 + 12 * 64 = 832$ bytes.

```
lean 02-4-2-list_compreh-proofs.lean
ls -l proper_names-original.arr proper_names-sorted.arr
```

The same `quicksortIndexed` program sorted both the ordinary arrays and the file array. The comparison includes the space padding, which does not change the order of these proper names. Duplicate randomly generated names are retained.